

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЖИТОМИРСЬКА ПОЛІТЕХНІКА
Кафедра ММСА

ЛАБОРАТОРНА РОБОТА № 2

з дисципліни «Моделювання та аналіз програмного забезпечення»

**ПОБУДОВА ДІАГРАМИ КЛАСІВ
ПРЕДМЕТНОЇ ОБЛАСТІ**

*Варіант предметної області: «Півот» — вебсервіс і браузерне розширення
для вивчення іноземної мови на власному контенті користувача*

Виконав: студент групи ВТ-23-2
Камінський Олексій Дмитрович

Перевірив: Власенко О.В.

Житомир – 2026

Репозиторій з кодом роботи:

<https://github.com/vt232kod-debug/software-modeling>

Мета

Варіант предметної області: «Півот» — вебсервіс і браузерне розширення для вивчення іноземної мови на основі власного контенту користувача. Роботу виконав студент групи ВТ-23-2 Камінський Олексій Дмитрович.

Опанувати нотацію діаграми класів UML (Unified Modeling Language — уніфікована мова моделювання): навчитися коректно моделювати класи, атрибути, методи та всі базові типи відношень — асоціацію з множинністю, агрегацію, композицію, узагальнення й реалізацію інтерфейсу — для предметної області, зафіксованої в Лабораторній роботі №1.

Предметна область спільна для всіх чотирьох лабораторних робіт курсу й для курсової роботи, і збігається з темою дипломної роботи. Межі моделі, прийняті в ЛР1, діють без змін: відеоплатформа розглядається одна — YouTube, а джерело субтитрів заховане за інтерфейсом ДжерелоСубтитрів, тому інші площадки присутні лише як точка підстановки з позначкою «інші адаптери поза межами роботи»; штучний інтелект — зовнішній сервіс за API, тому в моделі класів він лишає по собі не класи, а сліди роботи (вектори, перекази, теми словника); мобільний застосунок і Safari-версія — поза межами роботи; телеметрії й аналітики в системі свідомо немає.

Одну межу ця робота уточнює окремо, бо вона стосується саме моделі класів: **матеріал-вебсторінка не отримує власного класу-нащадка**. Стаття, додана посиланням, входить у модель як Документ з форматом «стаття-html» (UC12) і розбирається реалізацією ПарсерСтатті; третій нащадок Матеріал для вебсторінки свідомо лишений поза межами роботи, бо він не додав би жодного нового типу відношення — лише третій набір тих самих атрибутів. Тому формулювання «рівно два нащадки» далі в тексті означає «два нащадки в межах цієї моделі», а не «два види матеріалу в системі».

Якщо ЛР1 відповідала на питання «що система робить і для кого», то ця робота відповідає на питання «з чого система складається»: які сутності вона зберігає, що саме вона про них знає і як вони пов'язані життєвим циклом. Це перехід від моделі вимог до моделі даних, і головна перевірка якості тут — трасованість: кожен клас мусить мати походження в конкретному варіанті використання або в текстовій специфікації ЛР1.

Розділ 1. Від Use Case до класів: як виділялися сутності

Класи не вигадувалися наново. Джерелом був єдиний перелік із ЛР1: 18 варіантів використання, три повні текстові специфікації (UC04, UC05, UC12) і таблиця відкинутих кандидатів. Застосовано три послідовні фільтри.

Фільтр 1 — іменник зі специфікації, що має власний стан. З основних і альтернативних потоків UC04, UC05 і UC12 виписано всі іменники. Далі для кожного ставилося питання: чи система **пам'ятає** його між сеансами? «Доріжка реплік» — пам'ятає, бо кешує за відбитком: отже, Транскрипт. «Режим показу субтитрів» — пам'ятає, але це значення налаштування, а не сутність: отже, рядок НалаштуванняКористувача, а не клас. «Крок назад клавішею» — не пам'ятає взагалі: отже, не клас і не атрибут, а поведінка інтерфейсу.

Фільтр 2 — власний життєвий цикл. Іменник ставав класом, лише якщо він створюється й видаляється окремо від інших. СтанПовторення має власний життєвий цикл усередині життя картки слова, тому це клас, пов'язаний композицією, а не група полів у ЗбереженеСлово: стан перераховується після кожної вправи, і його зручно читати окремо від самої картки. Навпаки, Реплика власного життєвого циклу не має — вона народжується й помирає разом із доріжкою, не має ідентичності й не посилається ні на що, — тому лишилася типом значення в атрибуті Транскрипт.репліки: Реплика[].

Фільтр 3 — сутність, а не процес. Назви на «-ер», «-ор», «-менеджер», «-сервіс» на діаграмі класів предметної області не місце: це компоненти, і їхнє місце — ЛР3. Єдиний виняток свідомо зроблено для двох інтерфейсів і їхніх реалізацій (ДжерелоСубтитрів, ПарсерДокумента): методичка прямо вимагає показати реалізацію інтерфейсу, а саме ці дві точки підстановки є головними архітектурними рішеннями теми й безпосередньо породжують класи предметної області — Транскрипт і Документ відповідно.

Висновок: три фільтри дали 18 класів предметної області (серед них один абстрактний і один резервний), два інтерфейси й три класи оточення інтерфейсів — дві реалізації та фабрику; усе, що фільтри відкинули, не зникло, а перейшло в атрибути, у типи значень або в майбутні компоненти ЛР3.

Розділ 2. Перелік класів предметної області

Методичка вимагає щонайменше 6–8 класів, включно з одним абстрактним класом і одним інтерфейсом. Фактично побудовано **23 елементи**: 17 основних класів

предметної області (серед них обов’язковий абстрактний матеріал), 1 резервний клас, 2 інтерфейси, 2 класи-реалізації і 1 фабрику.

№	Елемент	Тип	Призначення
1	`Користувач`	Клас	Власник усіх особистих даних: акаунт, його словник, матеріали, профіль лексики, тариф
2	`НалаштуванняКористувача`	Клас	Налаштування як пара «ключ — значення»: одне джерело істини для вебзастосунку й розширення
3	`Матеріал`	Абстрактний клас	Будь-яка одиниця контенту в історії: те спільне, за що користувач бачить матеріал у списку
4	`Відео`	Клас (нащадок `Матеріал`)	Відеоматеріал платформи з тривалістю й позицією перегляду в мілісекундах
5	`Документ`	Клас (нащадок `Матеріал`)	Власний текст користувача в читалці: формат, вміст, сторінки, відпечаток
6	`ЗбереженеСлово`	Клас	Картка слова, збережена з матеріалу разом із реплікою-контекстом і мовною парою
7	`ТемаСловника`	Клас	Смислова тема словника, запропонована ШІ; службова тема «Різне» для негрупованого
8	`ВідомеСлово`	Клас	Запис профілю відомої лексики, окремого для кожної мови
9	`СтанПовторення`	Клас	Стан картки в розписанні інтервальних повторень (FSRS)
10	`ОцінкаРівня`	Клас	Результат тесту обсягу словника: « $\approx N$ слів»,

			похибка, рівень CEFR
11	`Підписка`	Клас	Платний стан акаунта: провайдер, зовнішній ідентифікатор, статус, кінець періоду
12	`ЛічильникВикористання`	Клас	Денна витрата лімітів за видами (переклад, пояснення, розбір, переказ, розпізнавання)
13	`СловниковаСтаття`	Клас	Спільна стаття довідника: лема, парамов, значення, частини мови
14	`Транскрипт`	Клас	Доріжка реплік відео. Спільно зберігається доріжка, зроблена розпізнаванням; доріжку, віддану самою платформою, розширення читає в межах сторінки
15	`РоботаРозпізнавання`	Клас	Замовлення доріжки в сервісу ШІ: стан, етап, кількість спроб, тривалість звуку (UC10)
16	`КешПерекладу`	Клас	Спільний переклад доріжки за відбитком «мови + рядки»: один переклад обслуговує всіх глядачів ролика
17	`КешПояснень`	Клас	Спільна відповідь ШІ за відбитком запиту: пояснення слова в контексті (UC08) і розбір речення (UC09)
18	`ПереказДоріжки`	Клас (резервний)	Спільний переказ доріжки за відбитком її рядків
19	`ДжерелоСубтитрів`	Інтерфейс	Контракт платформеного адаптера: плеєр, навігація, доріжка, розкладка панелі
20	`ДжерелоСубтитрівYou`	Клас-реалізація	Єдина реалізація `ДжерелоСубтитрів` у

	Tube`		межах роботи
21	`ФабрикаДжерелСубти трів`	Клас	Обирає реалізацію за платформою (фабричний метод до стратегії)
22	`ПарсерДокумента`	Інтерфейс	Контракт розбору файла в `Документ`: підтримка формату, розбір, пагінація, мова
23	`ПарсерPDF`	Клас-реалізація	Єдиний парсер, що дає справжні межі сторінок оригіналу

Клас ПереказДоріжки в моделі предметної області був позначений як резервний, і його свідомо внесено на діаграму: без нього з таблиці відношень зникає асоціація «Транскрипт — ПереказДоріжки». Разом із Транскрипт, СловниковаСтаття, РоботаРозпізнавання, КешПерекладу і КешПояснень він утворює шостий за розміром, але найцікавіший змістовно пакет — спільні довідники без власника, який окремо пояснено в Розділі 6.1.

Три класи — РоботаРозпізнавання, КешПерекладу і КешПояснень — додано до моделі саме тому, що без них варіанти використання UC08, UC09 і UC10 трасувалися б лише на методи й лічильники, тоді як їхній результат система **пам'ятає** між сеансами, а це і є Фільтр 1 з Розділу 1. КешПояснень свідомо зведено в один клас для двох видів розбору (пояснення слова й будова речення), бо відрізняються вони лише складом ключа й текстом запиту, а не типами відношень.

2.1. Абстрактний клас `Матеріал` і чому він абстрактний

Матеріал оголошено абстрактним, бо матеріал ніколи не існує «сам по собі»: у системі він завжди або **Відео** (є платформа, зовнішній ідентифікатор, тривалість, позиція в мілісекундах), або **Документ** (є формат, вміст, межі сторінок, відпечаток вмісту). Спільним лишається рівно те, за що користувач бачить матеріал у своїй історії: назва, мова контенту, карта частот слів, оцінка зрозумілості за його профілем лексики й позиція читання.

Ключовим є абстрактний метод джерелоТексту(): Iterable<Реплика>. У відео це репліки з часовими мітками, у документі — сторінки, розбиті на речення, але вся решта системи працює з ними однаково: оцінка зрозумілості, виділення фрагмента, збереження слова з контекстом, розбір речення. Якби Матеріал не був абстрактним, довелося б або дублювати обчислення зрозумілості у двох класах,

або створювати «просто матеріал» без джерела тексту — об'єкт, із яким нічого не можна зробити.

Це узагальнення не є теоретичною красою: воно **узгоджене** зі схемою даних, де обидва види лежать в одній таблиці історії з перелічуванням платформи, а документ додатково має власний рядок з деталями формату й вмісту. Слово «узгоджене», а не «тотожне», тут вжите свідомо: схема знає третє значення платформи для сторінки-статті, і модель приймає її як документ з форматом «стаття-html», а не як третього нащадка — межу зафіксовано в розділі «Мета». Друге розходження того самого роду — кратності з боку власника: у схемі власник рядка історії може бути ще не прив'язаний, і модель показує це кратністю 0..1 (Розділ 6).

Нащадків у Матеріал рівно два, і підставою є не «у системі лише два види контенту», а правило поділу: нащадок з'являється там, де відрізняється **реалізація** `джерелоТексту()`. У відео джерелом є доріжка реплік із часовими мітками, у документі — розбитий на сторінки текст. Стаття за посиланням дає рівно той самий текст, що й завантажений файл, тому власної реалізації не потребує й нащадком не стає.

2.2. Два інтерфейси й чому саме вони

ДжерелоСубтитрів — головне архітектурне рішення теми дипломної роботи. Інтерфейс оголошує шість операцій: `знайтиПлеєр()`, `спостерігатиНавігацію()`, `сховатиРодніСубтитри()`, `отриматиДоріжку()`, `побудуватиДжерелоСлова()`, `розкладкаПанелі()`. Реалізацію обирає фабрика `джерелСубтитрів` за платформою — це патерн Strategy з фабричним методом. Уся решта розширення платформенно-сліпа: оверлей, переклад і сховище знають лише інтерфейс. Саме тому обмеження «лише YouTube» у цій роботі — архітектурне рішення про межі моделі, а не вимушене спрощення: `ДжерелоСубтитрівВласнийФайл` (імпорт файлу `.srt`, `.vtt`, `.ass`, `.ttml`), `ДжерелоСубтитрівРозпізнане` (доріжка, зроблена ШІ, — UC10) та адаптери інших платформ входять тим самим каналом. Серверний бік UC10 — це окремий клас `РоботаРозпізнавання`: саме він зберігає стан замовлення, тоді як `ДжерелоСубтитрівРозпізнане` лише читає готову доріжку тим самим контрактом і тому на повну діаграму не внесений.

ПарсерДокумента — друга точка підстановки, що безпосередньо походить із кроків 4–5 основного потоку UC12: «система обирає парсер за розширенням і MIME-типом файла», «парсер витягує текст і, для PDF, справжні межі сторінок оригіналу». Інтерфейс оголошує `підтримує()`, `розібрати()`, `розбитиНаСторінки()`,

визначити `Мову()`, а реалізації — ПарсерPDF, ПарсерEPUB, ПарсерDOCX, ПарсерFB2, ПарсерПростогоТексту (TXT, MD і вставлений текст), ПарсерСтатті (HTML за посиланням). Новий формат додається без жодної зміни в читалці — саме це й перевіряє альтернативний потік 4а UC12 «формат не підтримується жодним парсером».

Два інтерфейси, а не один, узято тому, що вони реалізують різні контракти в різних частинах системи: перший живе в браузері користувача, другий — на сервері. Жоден із них не є узагальненням другого, і спільного предка в них немає — як і в прикладі методички, де інтерфейс оцінюваний реалізують непов'язані спадкуванням класи.

2.3. Що свідомо не стало класом

Кандидат	Чому не клас	Куди перейшов
`Реплика`	Тип значення без ідентичності: народжується й помирає разом із доріжкою, не посилається ні на що й не змінюється окремо	Атрибут `Транскрипт.реплики: Реплика[]`, тип повернення `реплікаНаЧас(мс)` і `джерелоТексту()`
`КарткаСлова` (UI-картка з UC05)	Екранне подання, а не сутність: та сама картка збирається зі `СловниковаСтаття` або з машинного перекладу й нічого не зберігає	Методи `ЗбереженеСлово.озвучити()`, `СловниковаСтаття.головніПереклади()`
`МенеджерСловника`, `СервісПерекладу`, `КонтролерЧиталки`	Процеси, а не сутності предметної області: у них немає стану, який система пам'ятає	Компоненти ЛРЗ («Модуль навчання», «Шар зовнішніх сервісів», «Веб-інтерфейс»)
`Мова`	Довідник із кількох десятків нерухомих рядків; окремий клас не додає ні поведінки, ні зв'язків	Тип-код `КодМови` в атрибутах `моваКонтенту`, `моваДжерела`, `моваЦілі`, `мова`
`ДенніЛіміти` як окремий клас	Ліміт — не запис, а функція тарифу й виду витрати	Методи `Підписка.стеля(вид)` і `ЛічильникВикористання.залишок(тариф)`
`ІсторіяМатеріалів`	Колекція, а не сутність: списком керує кратність `0..*` на композиції	Композиція `Користувач` — Матеріал з роллю «історія матеріалів»

Висновок: перелік містить 23 елементи діаграми, серед них обов'язкові за методичкою абстрактний клас Матеріал і два інтерфейси; шість кандидатів свідомо відкинуто, бо вони були або типами значень, або процесами, або колекціями, а не сутностями з власним станом і життєвим циклом.

Розділ 3. Атрибути, методи й модифікатори видимості

Методичка вимагає щонайменше 3 атрибути з коректною видимістю й типом і 2 методи на клас. Вимогу витримано для всіх 21 класу діаграми: найскромніші за складом — ЛічильникВикористання, ФабрикаджерелСубтитрів і ПарсерPDF (по 3 атрибути й 2–3 методи), найповніші — ЗбереженеСлово і Транскрипт (по 7 атрибутів і 3 методи). Два інтерфейси атрибутів не мають за означенням — вони оголошують лише операції (6 і 4 відповідно).

3.1. Правило вибору видимості

Видимість проставлялася не навмання, а за трьома правилами, однаковими для всієї діаграми.

Приватний (`-`) — усе, що є технічним ідентифікатором або похідним станом.

Сюди потрапили всі `id: uuid` (крім `ЛічильникВикористання`, у якого технічного ідентифікатора немає зовсім: рядок ідентифікується парою «день + вид витрати» в межах власника, і обидва атрибути пари позначено стандартним UML-модифікатором `{id}`), усі службові дати (створений, додано, пройдено, відкрита, оновлено), а також обчислювані кеші: `Матеріал.картаЧастот`, `Матеріал.кількістьТокенів`, `ЗбереженеСлово.вектор`, `Документ.початкиСторінок`, `Документ.вміст`. Жодне з цих полів не має сенсу змінювати ззовні: карта частот перераховується при завантаженні матеріалу, вектор — при збереженні слова, межі сторінок — при розборі файлу. Окремо приватним зроблено `Користувач.поштаПідтверджена`: цей прапорець має право змінити лише сценарій підтвердження пошти.

Публічний (`+`) — усе, що користувач бачить або чим керує через інтерфейс.

назва, слово, переклад, контекст, формат, поточнаСторінка, статус, кінецьПеріоду, `sefr`, `обсягСловника`. Критерій простий: якщо значення з'являється на екрані в сценаріях ЛР1, воно публічне.

Захищений (`#`) — лише те, що успадковують нащадки абстрактного класу.

Такий атрибут на діаграмі рівно один: `Матеріал.id: uuid`. Він має бути доступним у відео й документ (обидва будують за ним посилання на запис історії), але не має бути доступним іззовні. Це й є практична ілюстрація різниці між `-` і `#`: приватний ідентифікатор `Користувач.id` нащадків не має й тому лишається приватним, а `Матеріал.id` без `#` зробив би успадкування безглуздим.

Методи всюди публічні (+): на рівні моделі предметної області показується контракт сутності, а не її внутрішня кухня. Абстрактний метод позначено явно — `+ {abstract} джерелоТексту(): Iterable<Реплика>;` PlantUML рендерить його курсивом, як і назву абстрактного класу.

3.2. Перелічувані типи, ужиті в атрибутах

Щоб не захаращувати діаграму, перелічувані типи (enum) не виносилися в окремі елементи, а використані як типи атрибутів. Нижче — їхні множини значень; усі вони походять із моделі предметної області та зі специфікацій ЛР1.

Тип	Значення	Де вжито
`ТарифEnum`	базовий, з підпискою	`Користувач.тариф()`, `ЛічильникВикористання.залишок()`,
`СтатусEnum`	пробна, активна, прострочена, скасована	`Підписка.статус`
`ВидВитратиEnum`	переклад, пояснення, розбір, переказ, розпізнавання, завантаження	`ЛічильникВикористання.вид`, `Підписка.стеля()`,
`ПлатформаEnum`	youtube, document (інші адаптери поза межами роботи)	`Відео.платформа`, `Транскрипт.платформа`, `ФабрикаДжерелСубтитрів`
`ФорматEnum`	pdf, epub, docx, fb2, txt, md, стаття-html	`Документ.формат`, `ПарсерДокумента.підтримує()`,
`ОдиницяEnum`	сторінка оригіналу, фіксована кількість знаків, абзац	`Документ.оддиницяСторінки`
`СтанEnum`	нова, вчиться, повторення, перевчається	`СтанПовторення.стан`
`ПоходженняEnum`	пряма відмітка користувача, виведене за частотним рангом	`ВідомеСлово.походження`
`ВидРозборуEnum`	пояснення слова в контексті, розбір будови речення	`КешПояснень.вид`
`АкцентEnum`	варіанти вимови для озвучки, обрані в мастері першого запуску	`ЗбереженеСлово.озвучити()`,
`СтанРоботиEnum`	у черзі, звук, модель, запис, готово, помилка	`РоботаРозпізнавання.стан`
`ФорматСубтитрівEnum`	srt, vtt, ass, ttml	`ДжерелоСубтитрівВласнийФайл.формат` — клас поза повною діаграмою, показаний сірим лише на частковому виді 3

Два перелічення знадобляться вже в наступних роботах: `СтанEnum` і `СтанРоботиEnum` — це готові набори станів для діаграм станів картки слова й роботи з розпізнавання мовлення. Усі перелічення, крім `ФорматСубтитрівEnum`, ужиті в атрибутах класів, що справді присутні на повній діаграмі (рисунок 1); `ФорматСубтитрівEnum` належить класові поза межами роботи й тому позначений окремо.

Висновок: усі 21 клас діаграми мають щонайменше 3 атрибути й 2 методи, видимість проставлено за трьома явними правилами, а єдиний захищений атрибут `Матеріал.id` показує, навіщо модифікатор `#` взагалі існує.

Розділ 4. Повна діаграма класів

Діаграму побудовано в PlantUML, файл `classes.puml`; рендеринг — локальна збірка Graphviz. Елементи згруповано в п'ять пакетів, що відповідають природним підсистемам: «Акаунт і доступ», «Матеріали користувача», «Словник і прогрес», «Спільні довідники без власника», «Підстановка джерел і парсерів». Пакети — лише засіб читабельності; жодного відношення між пакетами в UML-сенсі не оголошено.

На діаграмі присутні **усі п'ять типів відношень** із розділу 1.2 методички: 9 композицій, 2 агрегації, 7 асоціацій з явною множинністю, 2 узагальнення й 2 реалізації інтерфейсу, а також 3 залежності — разом 25 зв'язків.

```
@startuml
' Лабораторна робота №2. Діаграма класів предметної області
' Предметна область: «Півот» – вебсервіс і браузерне розширення
' для вивчення іноземної мови на основі власного контенту користувача
' Камінський Олексій Дмитрович, група ВТ-23-2

skinparam defaultFontName "Helvetica"
skinparam defaultFontSize 15
skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam packageStyle rectangle
skinparam ArrowFontSize 13
skinparam nodesep 22
skinparam ranksep 55
skinparam class {
    BackgroundColor #FEFEFE
    BorderColor #33668E
    ArrowColor #33668E
}
skinparam package {
    BackgroundColor #F7FAFC
    BorderColor #9AB3C6
    FontStyle bold
}
skinparam interface {
    BackgroundColor #FFF8EC
    BorderColor #B07A28
}
skinparam note {
    BackgroundColor #FFFDF2
    BorderColor #C9B47A
}

' =====
' ПАКЕТ 1. Акаунт і доступ
' =====
package "Акаунт і доступ" {

    class Користувач {
        - id: UUID
    }
}
```

```

+ пошта: Email
+ ім'я: String
- поштаПідтверджена: Boolean
- створений: DateTime
+ тариф(): ТарифEnum
+ профільВідомих(мова: КодМови): Set<ВідомеСлово>
+ вивантажитиДані(): Архів
}

class НалаштуванняКористувача {
- id: UUID
+ {field} ключ: String(64)
+ значення: JSON
- оновлено: DateTime
+ встановити(ключ: String, значення: JSON): void
+ секретний(): Boolean
}

class Підписка {
- id: UUID
+ провайдер: String
+ зовнішнійId: String
+ статус: СтатусEnum
+ кінецьПеріоду: DateTime
+ активна(): Boolean
+ пробна(): Boolean
+ стеля(вид: ВидВитратиEnum): Int
}

class ЛічильникВикористання {
+ день: Date {id}
+ вид: ВидВитратиEnum {id}
+ кількість: Int
+ списати(n: Int): void
+ залишок(тариф: ТарифEnum): Int
+ скиданняЧерез(): Duration
}
}

' =====
' ПАКЕТ 2. Матеріали користувача
' =====
package "Матеріали користувача" {

abstract class Матеріал {
# id: UUID
+ назва: String
+ моваКонтенту: КодМови
+ зрозумілість: Real
- картаЧастот: Map<String,Int>
- кількістьТокенів: Int
+ перерахуватиЗрозумілість(профіль: Set<ВідомеСлово>): Real
+ позиціяЧитання(): Progress
+ {abstract} джерелоТексту(): Iterable<Реплика>
}

class Відео {
+ платформа: ПлатформаEnum
+ зовнішнійId: String
+ тривалістьMs: Long
+ останняПозиціяMs: Long
+ джерелоТексту(): Iterable<Реплика>
+ посиланняНаМомент(ms: Long): URL
}
}

```

```

class Документ {
    + формат: ФорматEnum
    - вміст: Text
    - початкиСторінок: Int[]
    + відпечаток: SHA256
    + поточнаСторінка: Int
    + одиницяСторінки: ОдиницяEnum
    + джерелоТексту(): Iterable<Реплика>
    + сторінка(n: Int): Text
    + змінитиМову(код: КодМови): void
}
}

' =====
' ПАКЕТ 3. Словник, профіль лексики та прогрес
' =====
package "Словник і прогрес" {

    class ЗбереженеСлово {
        - id: UUID
        + слово: String
        + переклад: String
        + контекст: String
        + моваДжерела: КодМови
        + моваЦілі: КодМови
        - {field} вектор: Vector(768)
        + озвучити(акцент: АкцентEnum): Audio
        + схожіЗаЗмістом(n: Int): List<ЗбереженеСлово>
        + прикріпитиДоТеми(тема: ТемаСловника): void
    }

    class ТемаСловника {
        - id: UUID
        + назва: String
        + піктограма: String
        + службова: Boolean
        - відкрита: DateTime
        + прогрес(): Real
        + нові(): Int
        + перейменувати(назва: String): void
    }

    class ВідомеСлово {
        - id: UUID
        + слово: String
        + мова: КодМови
        + походження: ПоходженняEnum
        - додано: DateTime
        + скасувати(): void
        + виведене(): Boolean
    }

    class СтанПовторення {
        - id: UUID
        + наступне: DateTime
        + стабільність: Real
        + складність: Real
        + повторень: Int
        + стан: СтанEnum
        + застосуватиВідповідь(результат: РезультатВправи): void
        + прострочене(): Boolean
        + наступнийІнтервал(): Duration
    }

    class ОцінкаРівня {
        - id: UUID
    }
}

```

```

+ мова: КодМови
+ обсягСловника: Int
+ cepr: String
- пройдено: DateTime
+ похибка(): Int
+ виведеніВідомі(корпус: ЧастотнийКорпус): List<String>
+ динаміка(): Delta
}
}

```

```

' =====
' ПАКЕТ 4. Спільні довідники без власника
' =====
package "Спільні довідники без власника" {

```

```

class Транскрипт {
- id: UUID
+ платформа: ПлатформаEnum
+ відеоId: String
+ мова: КодМови
+ репліки: Реплика[]
+ кількістьРеплік: Int
+ постачальник: String
+ реплікаНаЧас(мс: Long): Реплика
+ відпечаток(): SHA256
+ перекласти(цільоваМова: КодМови): Транскрипт
}

```

```

class РоботаРозпізнавання {
- id: UUID
+ платформа: ПлатформаEnum
+ відеоId: String
+ стан: СтанРоботиEnum
- етап: String
+ спроб: Int
+ тривалістьЗвукуМс: Long
+ просунути(стан: СтанРоботиEnum): void
+ завершена(): Boolean
+ причинаНеудачі(): String
}

```

```

class КешПерекладу {
- ключ: SHA256
+ моваДжерела: КодМови
+ моваЦілі: КодМови
+ переклади: String[]
+ кількістьРядків: Int
+ постачальник: String
+ знайти(ключ: SHA256): String[]
+ частковий(): Boolean
}

```

```

class КешПояснень {
- ключ: SHA256
+ вид: ВидРозборуEnum
+ запит: String
+ відповідь: JSON
+ постачальник: String
- востаннєВжито: DateTime
+ знайти(ключ: SHA256): JSON
+ застарілий(): Boolean
}

```

```

class ПереказДоріжки {
- ключ: SHA256
+ назва: String
}

```

```

    + переказ: JSON
    + кількістьРядків: Int
    + рядки(): List<String>
    + збігаєтьсяЗ(т: Транскрипт): Boolean
}

class СловниковаСтаття {
    - id: UUID
    + лема: String
    + параМов: String
    + важливість: Real
    + значення: Значення[]
    + головніПереклади(n: Int): List<String>
    + дляБудьЯкоїФорми(форма: String): Boolean
}

' =====
' ПАКЕТ 5. Точки підстановки: інтерфейси та їхні реалізації
' =====
package "Підстановка джерел і парсерів" {

    interface ДжерелоСубтитрів {
        + знайтиПлеєр(): Плеєр
        + спостерігатиНавігацію(обробник: Обробник): void
        + сховатиРодніСубтитри(прапор: Boolean): void
        + отриматиДоріжку(): Транскрипт
        + побудуватиДжерелоСлова(позиціяMs: Long): ДжерелоСлова
        + розкладкаПанелі(): Розкладка
    }

    class ДжерелоСубтитрівYouTube {
        - відеоId: String
        - адресаTimedtext: URL
        - автогенерована: Boolean
        - елементПлеєра: Плеєр
        + отриматиДоріжку(): Транскрипт
        + сховатиРодніСубтитри(прапор: Boolean): void
        .. решта операцій інтерфейсу ..
    }

    class ФабрикаДжерелСубтитрів {
        - реєстр: Map<ПлатформаEnum, Тип>
        - типоваПлатформа: ПлатформаEnum
        - активне: ДжерелоСубтитрів
        + створити(платформа: ПлатформаEnum): ДжерелоСубтитрів
        + зареєструвати(платформа: ПлатформаEnum, тип: Тип): void
    }

    interface ПарсерДокумента {
        + підтримує(формат: ФорматEnum): Boolean
        + розібрати(файл: Файл): Документ
        + розбитиНаСторінки(текст: Text): Int[]
        + визначитиМову(текст: Text): КодМови
    }

    class ПарсерPDF {
        - максимальнийРозмірМБ: Int
        - маєТекстовийШар: Boolean
        - межіСторінокОригіналу: Int[]
        + розібрати(файл: Файл): Документ
        + розбитиНаСторінки(текст: Text): Int[]
        .. решта операцій інтерфейсу ..
    }
}

```

```

' =====
' УЗАГАЛЬНЕННЯ (generalization)
' =====
Матеріал <|-- Відео
Матеріал <|-- Документ

' =====
' РЕАЛІЗАЦІЯ ІНТЕРФЕЙСУ (realization)
' =====
ДжерелоСубтитрів <|.. ДжерелоСубтитрівYouTube
ПарсерДокумента <|.. ПарсерPDF

' =====
' КОМПОЗИЦІЯ (composition) – спільний життєвий цикл з акаунтом
' =====
Користувач "0..1" *-- "1..*" НалаштуванняКористувача : налаштування
Користувач "1" *-- "0..1" Підписка : тариф
Користувач "1" *-- "0..*" ЛічильникВикористання : денні витрати
Користувач "0..1" *-- "0..*" Матеріал : історія матеріалів
Користувач "0..1" *-- "0..*" ЗбереженеСлово : особистий словник
Користувач "1" *-- "0..*" ТемаСловника : теми словника
Користувач "0..1" *-- "0..*" ВідомеСлово : профіль лексики
Користувач "0..1" *-- "0..*" ОцінкаРівня : історія тестів
ЗбереженеСлово "1" *-- "0..1" СтанПовторення : розписання

note bottom of Користувач
  **0..1, а не 1, у п'яти композиціях:**
  рядок, накопичений до першого входу,
  ще не має власника. Нуль – перехідний
  стан до прив'язки, а не другий власник.
  Де власник обов'язковий (підписка,
  лічильники, теми) – кратність 1.
  Докладно: Розділ 6, вступ.
end note

' =====
' АГРЕГАЦІЯ (aggregation) – частина не належить цьому цілому як композит
' =====
Матеріал "1" o-- "0..*" ЗбереженеСлово : джерело слова
note on link
  **Агрегація, а не композиція – компроміс.**
  Кратність 1: картки слова без матеріалу
  не буває. Частина цього цілого НЕ переживає:
  видалення матеріалу забирає й слова,
  й розписання. Ромб порожній лише тому, що
  композит у частини один – Користувач.
  Розділ 7, рядок 10.
end note

ТемаСловника "0..1" o-- "0..*" ЗбереженеСлово : згруповані слова

' =====
' АСОЦІАЦІЯ З КРАТНІСТЮ (association)
' =====
Відео "0..*" -- "0..1" Транскрипт : доріжка реплік >
Відео "0..*" -- "0..1" РоботаРозпізнавання : замовлене розпізнавання >
Транскрипт "1" -- "0..*" КешПерекладу : переклад доріжки >
Транскрипт "1" -- "0..1" ПереказДоріжки : переказ >
ЗбереженеСлово "0..*" -- "0..1" СловниковаСтаття : стаття довідника >
ЗбереженеСлово "0..*" -- "0..1" КешПояснень : пояснення в контексті >
ОцінкаРівня "0..1" -- "0..*" ВідомеСлово : поповнює профіль >

' =====

```


Часткові види не є іншими діаграмами: у них не змінено жодного класу, жодного атрибута й жодного типу відношення — показано підмножини тієї самої моделі у збільшеному масштабі. Класи, наведені як контекст, подані скорочено.

Одне розходження треба назвати прямо, щоб вид не прочитався як інша модель. Види 1 і 2 — **точні підмножини** рисунка 1. Вид 3 є підмножиною **плюс чотири класи, яких на повній діаграмі немає зовсім**: ДжерелоСубтитрівВласнийФайл, ДжерелоСубтитрівРозпізнане, ПарсерЕРУВ і ПарсерПростогоТексту. Вони позначені сірим і наведені не як частина моделі, а як доказ підстановності двох контрактів: саме на них видно, що реалізацій може бути скільки завгодно й що нова не змінює ні предка, ні клієнта. Разом із ними на виді 3 з'являються ще чотири реалізації — тобто шість замість двох на рисунку 1. Усі 23 елементи самої моделі покриті білими класами трьох видів без жодного пропуску.

5.1. Вид 1 — акаунт і його частини

Перший вид показує, що саме належить акаунту безпосередньо: налаштування, тариф, денні лічильники, історію тестів і профіль відомої лексики. Усі п'ять зв'язків тут — композиції: жодна з цих частин не має сенсу без власника й видаляється разом із ним.

Тут же видно два рішення про кратність, які легко прогавити. Перше: з боку Користувач у трьох із п'яти композицій стоїть $0..1$, а не 1 , — у схемі рядки, накопичені до першого входу, лишаються без власника й дістаються першому зареєстрованому, тож нуль є перехідним станом до прив'язки. У підписки й денних лічильників власник обов'язковий, і там збережено 1 . Друге: шостий зв'язок, ОцінкаРівня – ВідомеСлово, навмисно показано **асоціацією** з кратністю $0..1$ – $0..*$. Асоціацією — бо тест обсягу словника поповнює профіль лексики (це `<<include>> UC17 → UC07 з ЛР1`), але видалення запису тесту не повинне забирати знання користувача про слова. А $0..1$, а не 1 , — бо слово, позначене вручну в UC07, жодної оцінки рівня не має: це перше зі двох значень ПоходженняEnum, і воно ж є типовим.

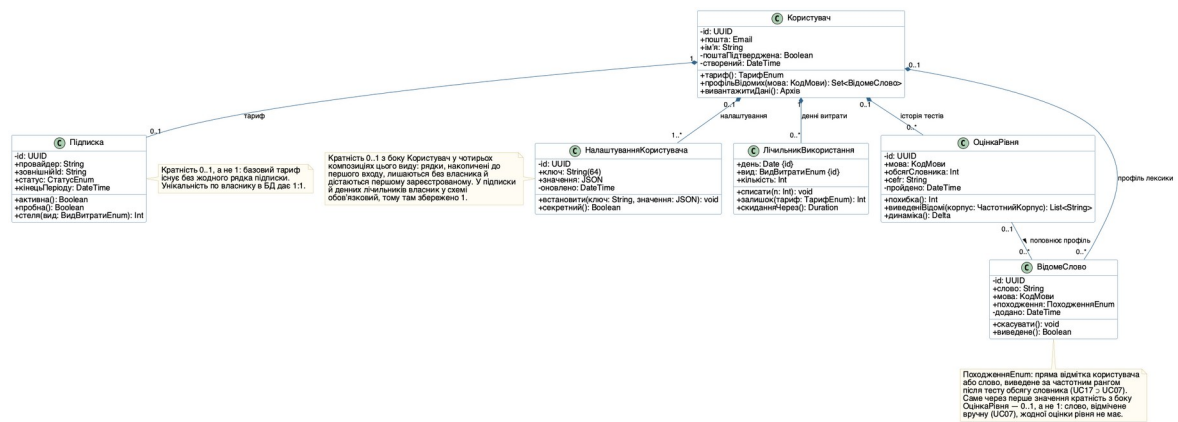


Рисунок 2 — Частковий вид 1: акаунт і його частини

5.2. Вид 2 — словник, повторення й спільний довідник

Другий вид показує найтоншу частину моделі: одне й те саме ЗбереженеСлово входить у три різні відношення «ціле-частина» одночасно. Від користувач — композиція з кратністю 0..1 з боку власника (слово видаляється разом з акаунтом, але до першого входу може бути ще безвласним). Від матеріал — агрегація з кратністю 1 (картка слова без матеріалу не створюється взагалі). Від Темасловника — агрегація з кратністю 0..1 (тема лише групує слова, і слово спокійно лежить без теми). Саме тут проходить головна межа між агрегацією й композицією, і саме тут ШІ-чернетка помилилася двічі (див. розділ «Критичний аналіз ШІ-чернетки»). Сьомий зв'язок виду — асоціація ЗбереженеСлово – КешПояснень: відповідь ШІ про слово в контексті спільна й без власника, тому повторний той самий запит не витрачає денної квоти (UC08, UC09).

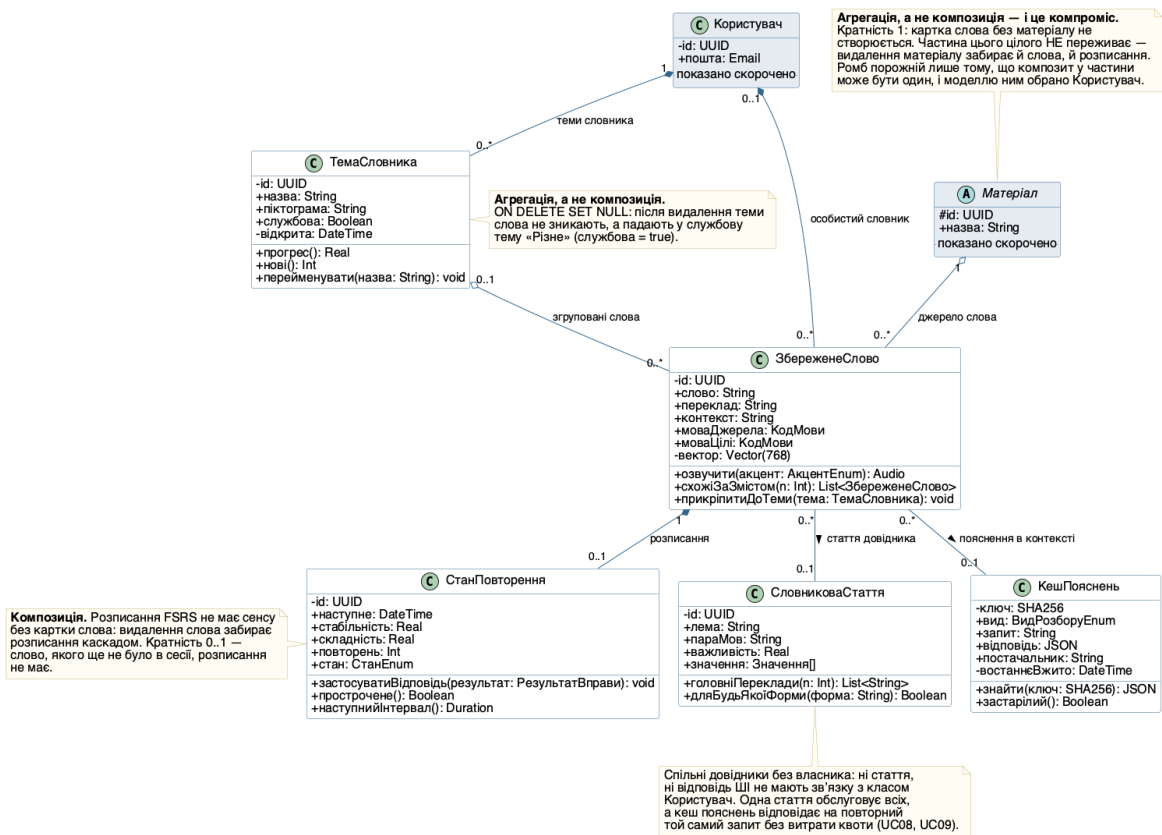


Рисунок 3 — Частковий вид 2: словник, повторення й спільний довідник

5.3. Вид 3 — матеріали, транскрипти й точки підстановки

Третій вид збирає разом обидва обов'язкові «ієрархічні» відношення: узагальнення (Матеріал $\leftarrow$ Відео, Матеріал $\leftarrow$ Документ) і реалізацію інтерфейсу. Реалізацій тут показано шість, а не дві, як на рисунку 1: чотири додаткові класи позначено сірим і вони, як сказано на початку розділу, **не входять до моделі** — вони наведені лише щоб було видно, що контракт справді підстановний. Цей вид найкраще показує, чому реалізація інтерфейсу — це **не** узагальнення: ДжерелоСубтитрівYouTube не є різновидом чогось, він лише обіцяє виконувати контракт, і між трьома реалізаціями немає жодного спільного стану.

Білою частиною виду, тобто самою моделлю, тут є ще два класи пакета спільних довідників: РоботаРозпізнавання (замовлення доріжки в сервісу ШІ з UC10 — стан, етап, кількість спроб) і КешПерекладу (переклад доріжки за відбитком «мови + рядки»). Обидва без власника, обидва ключовані вмістом або ідентифікатором ролика в площадки, і саме вони роблять твердження «другий глядач того самого ролика не платить» перевірним.

			мовну пару, тому `1..*`, а не `0..*`; уточнюється в UC03. Нуль з боку власника — безвласні рядки до першого входу
`Користувач` — `Підписка`	1 — 0..1	Композиція	UC18: базовий тариф існує без жодного рядка підписки; унікальність по власнику дає не більше однієї. Власник у схемі обов'язковий, тому `1`
`Користувач` — `ЛічильникВикористання`	1 — 0..*	Композиція	UC04, UC05, UC12: один рядок на пару «день + вид витрати». Власник обов'язковий, тому `1`
`Користувач` — `Матеріал`	0..1 — 0..*	Композиція	UC04, UC12: історія матеріалів акаунта; нуль — рядок історії, накопичений до входу
`Користувач` — `ЗбереженеСлово`	0..1 — 0..*	Композиція	UC06, UC14: особистий словник; нуль — слово, збережене до входу
`Користувач` — `ТемаСловника`	1 — 0..*	Композиція	UC14, UC15: теми словника особисті, бо групують особисті слова. Власник обов'язковий, тому `1`
`Користувач` — `ВідомеСлово`	0..1 — 0..*	Композиція	UC07: профіль відомої лексики, окремий для кожної мови; нуль — відмітка до входу
`Користувач` — `ОцінкаРівня`	0..1 — 0..*	Композиція	UC17: історія зростання рівня; нуль — тест, пройдений до входу
`ЗбереженеСлово` — `СтанПовторення`	1 — 0..1	Композиція	UC16: слово, яке ще не потрапляло в сесію, розписання не має; посилання на слово обов'язкове
`Матеріал` — `ЗбереженеСлово`	1 — 0..*	Агрегація	UC06: картка слова завжди зберігається з матеріалу, і посилання на нього обов'язкове,

			тому `1`, а не `0..1`. Ромб порожній лише тому, що композит у частини може бути один, і моделлю ним обрано `Користувач` (див. Розділ 7, рядок 10)
`ТемаСловника` — `ЗбереженеСлово`	0..1 — 0..*	Агрегація	UC15: нерозіbrane слово лежить без теми, у «Різному»
`Відео` — `Транскрипт`	0..* — 0..1	Асоціація	UC04: доріжка спільна за ключем «платформа + ідентифікатор відео»
`Відео` — `РоботаРозпізнавання`	0..* — 0..1	Асоціація	UC10: одна робота на ролик, спільна для всіх, хто його відкрив; нуль — доріжка є й замовляти нічого
`Транскрипт` — `КешПерекладу`	1 — 0..*	Асоціація	UC04: на одну доріжку буває кілька перекладів — по одному на цільову мову й постачальника
`Транскрипт` — `ПереказДоріжки`	1 — 0..1	Асоціація	UC11: переказ спільний, ключ — відбиток рядків доріжки
`ЗбереженеСлово` — `СловниковаСтаття`	0..* — 0..1	Асоціація	UC05, крок 3: стаття довідника спільна на всіх користувачів
`ЗбереженеСлово` — `КешПояснень`	0..* — 0..1	Асоціація	UC08, UC09: відповідь ІІІ спільна за відбитком запиту; нуль — пояснення ще не замовляли
`ОцінкаРівня` — `ВідомеСлово`	0..1 — 0..*	Асоціація	UC17 ` <include>` UC07: тест поповнює профіль, але не володіє ним. `0..1`, а не `1`, бо слово з походженням «пряма відмітка користувача» (UC07) жодної оцінки рівня не має</include>
`Матеріал` <— `Відео`	—	Узагальнення	Нащадок з'являється

			там, де відрізняється реалізація `джерелоТексту(): у відео це доріжка реплік із часовими мітками
`Матеріал` <— `Документ`	—	Узагальнення	У документі `джерелоТексту()` дає сторінки, розбиті на речення. Стаття за посиланням тим самим методом не відрізняється, тому входить як `Документ` з форматом «стаття- html», а не як третій нащадок (межі роботи, розділ «Мета»)
`ДжерелоСубтитрів` <… `ДжерелоСубтитрівYouTube`	—	Реалізація	UC04, крок 3: звернення до сервісу субтитрів YouTube
`ПарсерДокумента` <… `ПарсерPDF`	—	Реалізація	UC12, кроки 4–5: вибір парсера за форматом
`Відео` >… `ДжерелоСубтитрів`	—	Залежність	Відео читає доріжку лише через інтерфейс, не знаючи реалізації
`ФабрикаДжерелСубтитрів` >… `ДжерелоСубтитрів`	—	Залежність `<<create>>`	Підстановка реалізації за платформою
`ПарсерДокумента` >… `Документ`	—	Залежність `<<create>>`	Документ створюється парсером відповідного формату

6.1. Чому шість класів навмисно не пов’язані з `Користувач`

Увесь пакет «Спільні довідники без власника» — Транскрипт, РоботаРозпізнавання, КешПерекладу, КешПояснень, ПереказДоріжки і СловниковаСтаття — не має жодної асоціації з класом Користувач. Це не пропуск, а рішення, прийняте одночасно з міркувань приватності й економії.

Приватність. Ці класи ключуються вмістом або ідентифікатором ролика в площадки: доріжка — парою «платформа + ідентифікатор відео», переклад доріжки — відбитком «мови + рядки», пояснення — відбитком «слово + контекст + мови», переказ — відбитком рядків доріжки, стаття — лемою й парою мов. Поля «хто це дивився» в записі немає й не може бути. Отже, навіть маючи повний

доступ до цих таблиць, неможливо відновити, що саме конкретна людина дивилася чи читала. Рядок каже лише, що ролик хтось розшифровував, — і не каже хто.

Економія. Кратність $\Theta..*$ з боку Відео в асоціаціях з Транскрипт і РоботаРозпізнавання означає буквально таке: другий глядач того самого ролика посилається на вже наявну доріжку й на вже виконану роботу. Він не витрачає ні грошей на зовнішній сервіс, ні власної денної квоти — що прямо описано постумовою UC04: «переклад доріжки збережений у спільному кеші й доступний наступним глядачам того самого відео без повторних витрат». Тут важливо бути точним у межах цього твердження: спільними є саме **розпізнавання й переклад**. Доріжку, яку віддає сама платформа, розширення читає в межах сторінки й на сервері не зберігає — тому для відео з готовими субтитрами економія стосується перекладу, а не самої доріжки.

Чому в пакеті саме шість класів. У схемі однотипних спільних кешів більше: окремо лежать кеш перекладу, кеш пояснень слова, кеш розборів речення, черга робіт розпізнавання, частотний кеш і журнал списань за рядки перекладу. У модель внесено шість, і вибір зроблено за правилом «новий клас має або дати варіанту використання власну сутність, або додати новий тип відношення». Саме тому пояснення слова й розбір речення зведені в один кешПояснень з атрибутом вид: ВидРозборуEnum: таблиці різні, бо різні ключі й промпти, але на рівні предметної області це той самий факт — «відповідь ШІ, яку система пам'ятає». Частотний кеш і журнал списань нових сутностей варіантам використання не дають і нових відношень не вносять, тому лишилися поза моделлю.

Висновок: на діаграмі представлено всі п'ять обов'язкових типів відношень, у кожному відношенні «ціле-частина» й у кожній асоціації явно зазначено множинність обох кінців, а шість класів-довідників навмисно лишені без власника — і це рішення, а не недогляд.

Розділ 7. Обґрунтування агрегації й композиції за тестом розділу 1.2.5

Методичка вимагає письмово обґрунтувати вибір для **кожного** відношення «ціле-частина». На діаграмі їх одинадцять: дев'ять композицій і дві агрегації. До кожного застосовано той самий тест: «Якщо видалити ціле, чи має сенс частині продовжувати існувати сама по собі?» — і додатково другий критерій із порівняльної таблиці методички: «Чи може частина належати кільком цілим одночасно?»

Один випадок із одинадцяти розходиться з першим критерієм, і це розходження в таблиці нижче не згладжено, а названо: у рядку 10 частина цілого **не** переживає, але позначено агрегацію — бо інакше той самий клас отримав би два суцільні ромби. Решта десять відповідей на тест прямі.

Кратність 0..1 з боку користувач у п'яти композиціях відповідей на тест не змінює. Тест питає, що станеться з частиною при видаленні цілого, — і відповідь усюди та сама: частина видаляється каскадом. Нуль описує інший момент життя рядка — проміжок до першого входу, коли власника ще немає; щойно він з'являється, рядок прив'язується назавжди й іншого власника отримати не може. Саме неможливість другого власника, а не обов'язковість першого, і робить відношення композицією.

№	Відношення	Обраний тип	Відповідь на тест «чи має сенс частині існувати без цілого»
1	`Користувач` ♦ `НалаштуванняКористувача`	Композиція	Ні. Рядок «мовна пара = en → uk» поза конкретним акаунтом не означає нічого: це не довідник налаштувань, а значення саме цієї людини. Видалення акаунта забирає всі рядки
2	`Користувач` ♦ `Підписка`	Композиція	Ні. Підписка — це стан акаунта в платіжного провайдера; без власника вона не має ні предмета, ні платника. При видаленні акаунта запис підписки видаляється разом із ним
3	`Користувач` ♦ `ЛічильникВикористання`	Композиція	Ні. Запис «15 перекладів за 3 жовтня» без власника нічого не обмежує й нічого не описує — це не статистика сервісу, а особистий ліміт
4	`Користувач` ♦ `Матеріал`	Композиція	Ні. Запис історії — це «мій перегляд цього відео» з моєю позицією

			й моєю зрозумілістю. Саме відео на платформі далі існує, але запис історії без власника змісту не має
5	`Користувач` ♦ `ЗбереженеСлово`	Композиція	Ні. Картка слова містить мій контекст, мій переклад і моє розписання повторень; це особисті дані, що видаляються разом з акаунтом
6	`Користувач` ♦ `ТемаСловника`	Композиція	Ні. Тема групує мої слова: «Кулінарія» в моєму словнику й у чужому — це два різні набори. Без власника тема порожня за означенням
7	`Користувач` ♦ `ВідомеСлово`	Композиція	Ні. Профіль відомої лексики — це твердження «я знаю це слово»; поза акаунтом воно не є ні фактом про мову, ні фактом про когось
8	`Користувач` ♦ `ОцінкаРівня`	Композиція	Ні. Результат «≈4200 слів, B1» без того, чий він, — просто число. Історія тестів належить акаунту
9	`ЗбереженеСлово` ♦ `СтанПовторення`	Композиція	Ні. Розписання FSRS (наступна дата, стабільність, складність) описує саме цю картку. Видалення слова зі словника забирає розписання каскадом; «безхазяйний» стан повторення нічого не планує
10	`Матеріал` ◇ `ЗбереженеСлово`	Агрегація (свідомий компроміс)	Ні — і це треба сказати прямо. Частина цього цілого не переживає: видалення матеріалу з

			історії видаляє і його слова, і їхні розписання повторень, причому безумовно. Посилання на матеріал у схемі обов'язкове, звідси й кратність `1`. За першим критерієм методички це була б композиція. Агрегацію обрано виключно через другий критерій: частина може мати лише один композит, і моделлю ним обрано `Користувач` (рядок 5) — на рівні предметної області картка слова є перш за все особистими даними. Два суцільні ромби на одному класі зробили б модель суперечливою, тому з боку матеріалу ромб порожній, а фактичний каскад зафіксовано приміткою просто на зв'язку. Це єдине місце моделі, де тест методички й обране позначення розходяться, і розходження лишено видимим, а не прихованим
11	`ТемаСловника` ◇ `ЗбереженеСлово`	Агрегація	Так, і це перевірено поведінкою системи. При видаленні теми слова не зникають, а падають у службову тему «Різне» — у БД це `ON DELETE SET NULL`. Крім того, слово існує без теми взагалі: саме нерозібрані слова є

			входом для UC15 «Розкласти словник за темами». Отже, частина переживає ціле — агрегація
--	--	--	---

7.1. Чому жодне відношення не лишилося простою асоціацією «за замовчуванням»

Приклад методички окремо показує випадок Клієнт – Замовлення, де ромба не має бути взагалі: замовлення не є структурною частиною клієнта. У цій моделі такий самий статус мають усі сім асоціацій з Розділу 6. ЗбереженеСлово – СловниковаСтаття — найчистіший приклад: стаття довідника не є частиною картки слова, вона спільна для всіх і живе своїм життям; картка лише посилається на неї. Тут доречно уточнити два альтернативні потоки UC05, бо вони описують різні випадки: коли **прямої пари мов у довіднику немає**, стаття будується через англійську й у картці зазначається, що переклад непрямий (потік 3а), а коли **недоступний сам сервіс перекладу**, картка показує лише словникову статтю або повідомлення «переклад тимчасово недоступний» (потік 4а). Ні той, ні інший випадок не робить статтю частиною картки. Так само Відео – Транскрипт: доріжка не є частиною відео в акаунті, а спільним ресурсом, на який посилаються записи історії різних людей.

Висновок: усі одинадцять відношень «ціле-частина» обґрунтовано тестом методички. ТемаСловника ◇ ЗбереженеСлово — чиста агрегація: частина переживає ціле, у схемі це ON DELETE SET NULL, а слова падають у «Різне». Матеріал ◇ ЗбереженеСлово — агрегація за другим критерієм, а не за першим: частина не переживає, але композит у неї вже є, і це єдине свідомо лишене розходження моделі з тестом.

Розділ 8. Перевірка узгодженості з Лабораторною роботою №1

Трасованість перевірено на трьох рівнях: актори ЛР1 → елементи моделі класів, варіанти використання → задіяні класи й, найсуворіше, ключові іменники текстових специфікацій → класи або атрибути.

8.1. Актори ЛР1 на діаграмі класів

Головне, що треба зафіксувати: **актор і клас — різні речі**. Актор — це роль у взаємодії, клас — сутність із власним станом. Тому відображення не є взаємно однозначним, і кожне розходження має пояснення.

Актор ЛР1	Що йому відповідає в моделі	Пояснення
-----------	-----------------------------	-----------

	класів	
Користувач сервісу *(абстрактний)*	Не має власного класу	Його єдина спільна властивість — мовна пара й мова інтерфейсу — зберігається рядками `НалаштуванняКористувача`; абстрактним класом у цій моделі став `Матеріал`, а не користувач
Гість	Не має класу	До виконання UC01 у системі немає жодного запису; гість — це відсутність рядка `Користувач`
Зареєстрований користувач	`Користувач`	Пряме відображення: пошта, ім'я, підтвердження пошти, дата створення
Користувач з підпискою	Не окремий клас, а стан	`Користувач.тариф()` обчислюється з `Підписка.активна()`. Окремий клас-нащадок створив би два види користувачів у базі, хоча різниця між ними — один рядок у таблиці підписок
Адміністратор	Не має класу	Роль обслуговування, що діє поверх тих самих сутностей (`Підписка`, `ЛічильникВикористання`); окремих даних про себе система не зберігає
Зовнішній постачальник ШІ-функцій *(абстрактний)*	Не має класу	Зовнішня система не є сутністю нашої предметної області; у ЛРЗ вона стане компонентом «Шар зовнішніх сервісів»
Сервіс ШІ	Сліди роботи, не клас	`РоботаРозпізнавання`, `КешПояснень`, `Транскрипт.постачальник`, `ЗбереженеСлово.вектор`, `ПереказДоріжки`, `ТемаСловника`
Сервіс перекладу	Сліди роботи, не клас	`КешПерекладу`, `ЗбереженеСлово.переклад`, `Транскрипт.перекласти()`, `ЛічильникВикористання` з видом «переклад»
Сервіс субтитрів YouTube	`ДжерелоСубтитрівYouTube`	Єдиний актор, що отримав на діаграмі класів прямий відповідник — саме тому, що звернення до нього

		інкапсульоване інтерфейсом
Платіжний провайдер	Поля, не клас	`Підписка.провайдер`, `Підписка.зовнішнійId`, `Підписка.статус`
Провайдер авторизації	Не має класу	Запис зовнішньої автентифікації належить шару доступу (компонент «API сервісу та контроль доступу» в ЛРЗ), а не предметній області
Поштовий сервіс	Один прапорець	`Користувач.поштаПідтверджен а`; самі листи система не зберігає

8.2. Трасування «Use Case → задіяні класи»

Use Case ЛР1	Задіяні класи, атрибути й методи
UC01 Зареєструватися в сервісі	`Користувач` (`пошта`, `ім'я`, `створений`), `НалаштуванняКористувача.встановити()` — мовна пара й акцент у мастері першого запуску
UC02 Увійти в сервіс	`Користувач` (`пошта`, `поштаПідтверджена`)
UC02.1 Увійти поштою або паролем	`Користувач.поштаПідтверджена`
UC02.2 Увійти через зовнішній акаунт	`Користувач.пошта`
UC03 Налаштувати параметри вивчення	`НалаштуванняКористувача` (`ключ`, `значення`, `встановити()`), `секретний()` — для власного ключа III)
UC04 Дивитися відео з двомовними субтитрами	`Відео`, `Транскрипт` (`реплікаНаЧас()`), `перекласти()`, `відпечаток()`), `КешПерекладу` (`знайти()`, `частковий()`), `ДжерелоСубтитрів`, `ДжерелоСубтитрівYouTube`, `ФабрикаДжерелСубтитрів`, `Матеріал.перерахуватиЗрозумілість()`, `ЛічильникВикористання.списати()`
UC05 Отримати значення слова	`СловниковаСтаття` (`головніПереклади()`), `дляБудьЯкоїФорми()`), `КешПерекладу` — коли прямої статті немає, `Транскрипт.реплікаНаЧас()`, `ЗбереженеСлово.озвучити()`, `ЛічильникВикористання`
UC06 Зберегти слово у словник	`ЗбереженеСлово` (`слово`, `переклад`, `контекст`, `моваДжерела`, `моваЦілі`), агрегація від `Матеріал`
UC07 Позначити слово як відоме	`ВідомеСлово` (`походження`, `скасувати()`), `Користувач.профільВідомих()`
UC08 Отримати пояснення слова в контексті	`КешПояснень` (`вид` = пояснення слова, `знайти()`, `застарілий()`), `ЗбереженеСлово.контекст`

	`ЛічильникВикористання.залишок()`, `Підписка.стеля()``
UC09 Розібрати будову речення	`КешПояснень` (`вид` = розбір речення, `відповідь`), `Транскрипт.реплікаНаЧас()`, `Документ.сторінка()`, `Підписка.активна()``
UC10 Замовити розпізнавання мовлення для відео	`РоботаРозпізнавання` (`стан`, `спроб`, `просунути()`, `завершена()`, `причинаНеудачі()`), `Транскрипт.постачальник`, `Підписка.активна()``
UC11 Отримати переказ матеріалу	`ПереказДоріжки` (`переказ`, `рядки()`, `збігається3()`, `Матеріал.зрозумілість`, `перерахуватиЗрозумілість()`, `Підписка.активна()`, `Підписка.стеля(вид)``
UC12 Завантажити власний матеріал	`Документ` (`формат`, `вміст`, `відпечаток`, `змінитиМову()`), `ПарсерДокумента`, `ПарсерPDF`, `ПарсерEPUB`, `ПарсерПростогоТексту`, `Матеріал.картаЧастот`
UC13 Читати власний текст у читалці	`Документ` (`сторінка()`, `поточнаСторінка`, `початкиСторінок`, `одиницяСторінки`), `Матеріал.позиціяЧитання()``
UC14 Працювати зі словником за темами	`ТемаСловника` (`прогрес()`, `нові()`, `перейменувати()`), `ЗбереженеСлово`, `СтанПовторення.наступне`, `Користувач.вивантажитиДані()``
UC15 Розкласти словник за темами	`ЗбереженеСлово` (`вектор`, `схожіЗаЗмістом()`, `прикріпитиДоТеми()`), `ТемаСловника.службова` — тема «Різне», `Підписка.активна()`, `Підписка.стеля(вид)``
UC16 Пройти сесію інтервальних повторень	`СтанПовторення` (`застосуватиВідповідь()`, `прострочене()`, `наступнийІнтервал()`, `стан`), `ЗбереженеСлово`
UC17 Пройти тест обсягу словника	`ОцінкаРівня` (`обсягСловника`, `sefr`, `похибка()`, `виведеніВідомі()`, `динаміка()`), `ВідомеСлово.походження`
UC18 Оформити й керувати підпискою	`Підписка` (`статус`, `кінецьПеріоду`, `активна()`, `пробна()`, `стеля()`), `ЛічильникВикористання.залишок()``

Жоден із 18 варіантів використання не лишився без класів, і жоден клас не лишився без варіанта використання — перевірка пройдена в обидва боки. Усі класи, названі в таблиці, присутні на повній діаграмі (рисунок 1): жоден варіант використання не трасується на клас, якого в моделі немає. Окремо звірено гейт підписки: усі чотири платні варіанти використання, які в ЛР1 належать акторові

«Користувач з підпискою» (UC09, UC10, UC11, UC15), трасуються на Підписка.активна() або Підписка.стеля(вид) однаково.

8.3. Звіряння ключових іменників текстових специфікацій

Найсуворіша перевірка методички: кожен ключовий іменник із повних текстових специфікацій UC04, UC05 і UC12 має бути присутній серед класів або атрибутів. Нижче — повний перелік; іменники, що описували дію або екранний елемент, позначено окремо.

Іменник зі специфікації LP1	Де присутній у моделі класів
плеєр	`ДжерелоСубтитрів.знайтиПлеєр()`, `ДжерелоСубтитрівYouTube.елементПлеєра`
ідентифікатор відео	`Відео.зовнішнійId`, `Транскрипт.відеоId`
доріжка реплік	Клас `Транскрипт`
репліка	Тип значення `Реплика` в `Транскрипт.репліки` і `джерелоТексту`
мова оригіналу	`Матеріал.моваКонтенту`, `Транскрипт.мова`, `ЗбереженеСлово.моваДжерела`
переклад доріжки	`Транскрипт.перекласти(цільоваМова)`
спільний кеш	Класи `Транскрипт`, `КешПерекладу`, `КешПояснень`, `ПереказДоріжки` без власника (Розділ 6.1)
відбиток	`Транскрипт.відпечаток()`, `Документ.відпечаток`, `ПереказДоріжки.ключ`
денний ліміт	Клас `ЛічильникВикористання`, метод `залишок(тариф)`
профіль відомої лексики	Клас `ВідомеСлово`, метод `Користувач.профільВідомих(мова)`
слово	`ЗбереженеСлово.слово`, `ВідомеСлово.слово`
підсвічування незнайомих слів	Похідне від `профільВідомих()`; стану не має, тому не клас
режим показу субтитрів	Рядок `НалаштуванняКористувача`
панель транскрипту	`ДжерелоСубтитрів.розкладкаПанелі`
позиція перегляду	`Відео.останняПозиціяМс`, `Матеріал.позиціяЧитання`
зрозумілість матеріалу	`Матеріал.зрозумілість`, `перерахуватиЗрозумілість(профіль)`
історія матеріалів	Композиція `Користувач ♦— Матеріал` з кратністю `0..*`
тариф	`Користувач.тариф()`, клас `Підписка`
розпізнавання мовлення	Клас `РоботаРозпізнавання` (стан: СтанРоботиEnum); браузерний бік — `ДжерелоСубтитрівРозпізнане` на частковому виді 3

джерело перекладу	`Транскрипт.постачальник`, `КешПерекладу.постачальник`
речення	`Транскрипт.реплікаНаЧас()`, `Документ.сторінка()`
лема	`СловниковаСтаття.лема`
словникова стаття	Клас `СловниковаСтаття`
пара мов	`СловниковаСтаття.параМов`, `ЗбереженеСлово.моваДжерела` + `моваЦілі`
значення й частини мови	`СловниковаСтаття.значення: Значення[]`
машинний переклад	`ЗбереженеСлово.переклад`
картка слова	Екранне подання; збирається з `СловниковаСтаття` і `ЗбереженеСлово` (Розділ 2.3)
контекстна репліка	`ЗбереженеСлово.контекст`
озвучка й акцент	`ЗбереженеСлово.озвучити(акцент: АкцентEnum)`
парсер	Інтерфейс `ПарсерДокумента` та його реалізації
формат, розширення, MIME-тип	`Документ.формат: ФорматEnum`, `ПарсерДокумента.підтримує(формат)`
межі сторінок оригіналу	`Документ.початкиСторінок`, `ПарсерPDF.межіСторінокОригіналу`
одиниця сторінки	`Документ.одиницяСторінки: ОдиницяEnum`
карта частот слів	`Матеріал.картаЧастот: Map<String,Int>`
мова тексту, визначена евристикою	`ПарсерДокумента.визначитиМову()`, `Документ.змінитиМову()`
обмеження розміру файла	`ПарсерPDF.максимальнийРозмірМБ`
запис в історії матеріалів	Екземпляр `Відео` або `Документ` як нащадків `Матеріал`
дублікат за відбитком вмісту	`Документ.відпечаток: SHA256`
читалка	Методи `Документ.сторінка(n)`, `поточнаСторінка`

Жоден ключовий іменник не лишився без місця. Три позиції — «підсвічування», «картка слова» і «читалка» — присутні як поведінка, а не як дані, і це зафіксовано явно: вони описують подання, а не стан, який система пам'ятає.

8.4. Пакети діаграми класів і компоненти Лабораторної роботи №3

Пакети на діаграмі класів — засіб читабельності, а не компоненти. Щоб ця теза не виглядала відмовкою, нижче наведено фактичну відповідність до восьми компонентів ЛР3. Вона **неповна й не взаємно однозначна**: пакетів п'ять, компонентів вісім, і один пакет розходиться на два компоненти.

Пакет діаграми класів (ЛР2)	Компонент ЛР3	Характер відповідності
Акаунт і доступ	4. API сервісу та контроль	Розходиться на два:

	доступу + 5. Тарифи й ліміти	Користувач і Налаштування Користувача належать четвертому, Підписка й Лічильник Використання — п'ятому
Матеріали користувача	7. Модуль контенту	Пряма відповідність
Словник і прогрес	6. Модуль навчання	Пряма відповідність
Спільні довідники без власника	7. Модуль контенту (кеші) + 8. Шар зовнішніх сервісів (постачальники)	Розходиться: самі записи зберігає модуль контенту, а наповнює їх шар зовнішніх сервісів
Підстановка джерел і парсерів	1. Оверлей розширення + 2. Адаптер платформи YouTube (інтерфейс ДжерелоСубтитрів); 7. Модуль контенту (інтерфейс ПарсерДокумента)	Єдиний пакет, що перетинає межу «браузер — сервер»
— (класу не відповідає)	3. Веб-інтерфейс	Компонент подання: власних даних не має, тому класу предметної області йому не відповідає

Висновок: узгодженість із ЛР1 перевірено в трьох зрізах; усі 18 варіантів використання мають класи, усі ключові іменники специфікацій знайшли місце серед класів, атрибутів або методів, а кожне розходження між актором і класом пояснено окремо — актор є роллю, а не сутністю. Відповідність «пакет класів → компонент ЛР3» існує, але частково: саме тому пакети не названо попереднім поділом на компоненти.

Розділ 9. Відповіді на питання для самоконтролю

9.1. Чим відрізняється агрегація від композиції з погляду життєвого циклу об'єктів

Композиція означає спільний життєвий цикл: частина не може існувати без цілого й видаляється разом із ним, а належати вона може лише одному цілому. Агрегація означає, що ціле лише «має» частину: частина переживає зникнення цілого й може належати кільком цілим або міняти власника. Приклад композиції з цієї роботи — ЗбереженеСлово ♦ СтанПовторення: розписання повторень описує конкретну картку слова й разом з нею зникає. Приклад агрегації — ТемаСловника ◇ ЗбереженеСлово: видалення теми «Кулінарія» не видаляє слів, вони переходять у службову тему «Різне».

9.2. Що означає множинність `0..1` на асоціативному зв'язку

0..1 означає «нуль або один», тобто повністю опційний зв'язок: об'єкт або має рівно одного партнера, або не має жодного. У цій роботі така множинність стоїть на **чотирнадцяти** кінцях зв'язків — п'ять разів з боку Користувач у композиціях з pullable власником і дев'ять разів у решті відношень. Нижче — три найпоказовіші з чотирнадцяти. Користувач "1" *-- "0..1" Підписка: базовий тариф існує без жодного рядка підписки, тому нуль тут — нормальний стан, а не помилка даних. ЗбереженеСлово "1" *-- "0..1" СтанПовторення: слово, яке ще жодного разу не потрапляло в сесію повторень, розписання не має. ОцінкаРівня "0..1" -- "0..*" ВідомеСлово: слово, позначене відомим вручну в UC07, не походить із жодного тесту обсягу словника, і саме цей нуль відрізняє пряму відмітку від слова, виведеного за частотним рангом.

9.3. У чому різниця між узагальненням і реалізацією на діаграмі класів

Узагальнення (суцільна лінія з порожнім трикутником) — це відношення «є різновидом»: нащадок успадковує і структуру, і поведінку предка, і підмінює його всюди, де той очікується. Реалізація (пунктирна лінія з порожнім трикутником) — це відношення «виконує контракт»: клас зобов'язується мати оголошені операції, але не успадковує жодного поля. На цій діаграмі Відео є різновидом Матеріал — у нього є успадковані назва, зрозумілість, карта частот. А ДжерелоСубтитрівYouTube не є різновидом нічого: у трьох реалізацій ДжерелоСубтитрів немає жодного спільного поля, бо одна працює з мережевим запитом до платформи, друга — із завантаженим файлом, третя — із чергою розпізнавання.

9.4. Чому клас позначається курсивом і що це означає для створення екземплярів

Курсив (або стереотип {abstract}) означає, що клас абстрактний: власних екземплярів він не має й створений лише для успадкування. У цій моделі абстрактним є Матеріал: «просто матеріалу» в системі не існує, існують Відео і Документ. Практичний наслідок — у Матеріал є абстрактний метод джерелоТексту(), тіла якого на цьому рівні написати неможливо: у відео це репліки з часовими мітками, у документі — сторінки. Спроба створити «просто Матеріал» дала б об'єкт, у якого цей метод немає реалізації, тобто об'єкт, із яким не працює жоден сценарій ЛР1.

9.5. Яку роль відіграють модифікатори видимості і чому вони важливі вже на етапі моделювання

Видимість — це перший документ про те, що в сутності є контрактом, а що деталлю реалізації. На етапі моделювання вона дає три практичні відповіді.

Перша: що взагалі показувати в інтерфейсі — усе публічне в Підписка (статус, кінець періоду) справді бачить користувач, а приватний id — ні. Друга: що має право змінювати лише сама система — Матеріал.карта частот приватна, бо перераховується при завантаженні, і дозвіл редагувати її ззовні зробив би оцінку зрозумілості недостовірною. Третя: де пролягає межа спадкування — єдиний захищений атрибут Матеріал.id показує, що саме нащадки мають бачити, а що ні. Без цих рішень діаграма класів перетворюється на малюнок таблиць бази даних.

9.6. Які ризики виникають, якщо покластися на ІІІ-згенерований перелік класів без перевірки

Головний ризик — діаграма виглядатиме бездоганно, але описуватиме не цю систему. ІІІ не знає рішень, які не записані в тексті: що транскрипт спільний і без власника, що тема словника видаляється через SET NULL, що межі сторінок дає лише PDF. Тому його чернетка систематично помиляється у двох напрямках: вона робить усе «частиною» головного класу (бо так симетричніше) і перетворює процеси на класи з назвами на «-менеджер» (бо так виглядає солідніше). Обидві помилки непомітні на око й вилізуть лише на етапі проектування схеми бази даних. Конкретні шість помилок саме цієї чернетки й виправлення до них наведені в наступному розділі.

Висновок: відповіді спираються на власну модель, а не на приклад методички: кожне теоретичне означення проілюстроване відношенням, яке справді є на побудованій діаграмі.

Критичний аналіз ІІІ-чернетки

Чернетку діаграми класів згенеровано ІІІ-асистентом за текстовим описом предметної області й переліком варіантів використання ЛР1, після чого проведено ручну звірку з логікою сервісу, зі схемою даних і з означеннями розділу 1.2 методички. Нижче — шість реальних помилок саме цієї чернетки.

Помилка ІІІ-чернетки	Чому це помилка	Як виправлено	
Спільні довідники прив'язані до `Користувач` композицією. Чернетка послідовно зробила кожен клас «частиною» користувача, бо так симетричніше: і `Транскрипт`, і	Доріжка субтитрів, переклад доріжки, пояснення ІІІ й стаття довідника спільні: вони зберігаються за хеш-ключем вмісту, а не за власником. Композиція тут знищила б обидва сенси рішення —	Усі такі класи винесено в окремий пакет «Спільні довідники без власника» — `Транскрипт`, `РоботаРозпізнавання`, `КешПерекладу`, `КешПояснень`, `ПереказДоріжки`,	

«СловниковаСтаття» отримали суцільний ромб від акаунта	приватність (із записів неможливо дізнатися, хто що дивився) і економію (другий глядач того самого ролика не платить ні грошиками, ні квотою). Крім того, при видаленні акаунта каскад забрав би доріжку, якою користуються інші	«СловниковаСтаття»; зв'язки замінено на асоціації без ромба. Рішення пояснено в Розділі 6.1 і приміткою на самій діаграмі	
Обидві агрегації намальовано композиціями. ШІ побачив у описі «ON DELETE CASCADE» для «Матеріал» → «ЗбереженеСлово» і поставив суцільний ромб; для «ТемаСловника» → «ЗбереженеСлово» він вважав слова структурною частиною теми	Фізичний каскад у базі даних і композиція в UML — різні твердження. За порівняльною таблицею методички частина може належати лише одному цілому в композиції, а «ЗбереженеСлово» вже є частиною «Користувач»: два суцільні ромби на одному класі роблять модель суперечливою. У випадку теми тест методички дає ще й пряму відповідь: видалення теми не видаляє слів, а переводить їх у службову тему «Різне» («ON DELETE SET NULL»), і слово зазвичай спочатку існує без теми — нерозібрані слова є входом UC15. Композиція означала б, що словник не може містити негрупованих слів	Обидва зв'язки замінено на агрегації. Для теми це чиста агрегація з кратністю «0..1 — 0..*», а службову тему відображено атрибутом «ТемаСловника.служба: Boolean». Для матеріалу агрегацію лишено з кратністю «1 — 0..*» і з прямим зазначенням у Розділі 7 (рядок 10), що частина цілого не переживає й що ромб порожній лише через заборону двох композитів	
Кратність «1» з боку «цілого» скрізь, де є зовнішній ключ.	Обидві групи кратностей суперечать тому, як система	Виправлено на «0..1» у п'яти композиціях із nullable власником і на	

Чернетка поставила `1` у всіх вісьмох композиціях від `Користувач` і `1` з боку `ОцінкаРівня` в асоціації з `ВідомеСлово`	справді працює. У схемі власник рядка nullable в налаштуваннях, історії матеріалів, словнику, профілі лексики й історії тестів: рядки, накопичені до появи входу, лишаються без власника. `1` тут означало б, що таких рядків не буває, — а вони є, і механізм їх прив'язки описаний окремо. З `ОцінкаРівня` ще гірше: `1` читалося б як «кожне відоме слово походить із тесту», тоді як `ПоходженняEnum` має значення «пряма відмітка користувача», і це типовий шлях слова в профіль (UC07 — самостійний варіант використання)	`0..1` з боку `ОцінкаРівня`; там, де власник у схемі обов'язковий (підписка, лічильники, теми словника), `1` збережено. Пояснення — у вступі Розділу 6, у таблиці відношень і примітками на діаграмі та на виді 1	
Атрибути, яких у системі немає. Чернетка дописала `ОцінкаРівня.погрішність: Int` і `ЛічильникВикористання.id: UUID`, бо так «акуратніше»: у кожної сутності має бути `id`, а якщо в тексті згадано «$\pm N$ слів», то це начебто поле	Фільтр 1 цієї ж роботи питає, чи система пам'ятає значення. Похибка оцінки не зберігається: це фіксована константа дизайну («± 400 за базовою вибіркою, ± 200 після уточнення межі»), тобто похідне значення, а не стан. У лічильника витрат технічного ідентифікатора немає зовсім: рядок ідентифікується парою «день + вид витрати» в межах власника, що сам Розділ 6 і описує. Зайвий атрибут	`погрішність` прибрано з атрибутів і замінено методом `+ похибка()`: Int` — похідне значення. У `ЛічильникВикористання` знято `id`, а `день` і `вид` позначено стандартним UML-модифікатором `{id}` як ідентифікуючу пару; примітка на діаграмі це пояснює	

	небезпечніший за відсутній: він обіцяє поле, якого ніхто не заповнює		
`ДжерелоСубтитрів` — абстрактний клас, а `ДжерелоСубтитрівYouTube` — його нащадок через узагальнення. Чернетка не розрізняє реалізацію й спадкування	Трьох реалізацій не об'єднує жодне спільне поле: одна працює з мережевим запитом до платформи, друга — із завантаженням файлом `.srt`, третя — з чергою розпізнавання. Спільним є лише контракт, а підстановку виконує фабрика за платформою — це Strategy, а не ієрархія типів. Абстрактний клас тут нав'язав би нащадкам стан, якого в них немає, і зламав би можливість додати адаптер нової платформи без зміни предка	Оголошено `interface ДжерелоСубтитрів` з шістьма операціями; зв'язки замінені на реалізацію (`<	..`); додано `ФабрикаДжерелСубтитрів` із залежністю `<<create>>`. Абстрактним класом у моделі лишився `Матеріал`, де спільний стан справді є
Клас-процеси замість сутностей: `МенеджерСловника`, `СервісПерекладу`, `КонтролерЧиталки`, `ОбробникПідписки`. Чернетка видала їх із повними наборами методів	Жоден із них не має стану, який система пам'ятає між сеансами, — отже, на діаграмі класів предметної області їм не місце. Це функціональна декомпозиція, замаскована під модель даних: вона робить діаграму більшою, але не додає жодного факту про предметну область. Їхнє правильне місце — діаграма компонентів ЛРЗ	Усі чотири прибрано; їхню поведінку роздано сутностям як методи (`ЗбереженеСлово.схожістьЗЗмістом()`, `Транскрипт.перекласти()`, `Документ.сторінка()`, `Підписка.стеля()`), а самі назви перенесено в перелік майбутніх компонентів ЛРЗ	

Висновок: чернетка ІІІ дала корисний чорновий перелік сутностей за кілька хвилин, але жодне з шести виправлень не вимагало знання UML — усі шість вимагали знання саме цього сервісу та його схеми даних. Три помилки з шести —

про той самий нахил: усе зробити частиною користувача (зайві композиції, два суцільні ромби на одному класі, кратність 1 з боку власника). Найнебезпечнішою виявилася не плутанина агрегації з композицією, яку видно за тестом методички, а саме кратність: вона виглядає бездоганно й суперечить системі лише тоді, коли звірити її з конкретною колонкою схеми.

Висновки

У роботі побудовано діаграму класів предметної області «Півот» — вебсервісу й браузерного розширення для вивчення іноземної мови на основі власного контенту користувача. Діаграма містить 23 елементи: 17 основних класів предметної області (серед них обов'язковий абстрактний клас Матеріал), один резервний клас ПереказДоріжки, два інтерфейси ДжерелоСубтитрів і ПарсерДокумента, два класи-реалізації та одну фабрику. Кожен із 21 класу має щонайменше три атрибути з явною видимістю й типом і щонайменше два методи, що втричі перевищує мінімум методички в 6–8 класів. Оскільки повна діаграма на аркуші читається погано, додатково побудовано три часткові види: види 1 і 2 — точні підмножини повної діаграми, вид 3 — підмножина плюс чотири сірі класи реалізацій, які прямо названо такими, що в модель не входять.

Представлено всі п'ять типів відношень із теоретичної частини: дев'ять композицій, дві агрегації, сім асоціацій з явною множинністю обох кінців, два узагальнення й дві реалізації інтерфейсу, а також три залежності зі стереотипом <<create>> — разом 25 зв'язків. Кожне з одинадцяти відношень «ціле-частина» обґрунтовано письмово за тестом розділу 1.2.5 методички, і одне з одинадцяти свідомо лишено як видиме розходження: Матеріал $\diamond$ ЗбереженеСлово за першим критерієм було б композицією, бо видалення матеріалу справді забирає і слова, і їхні розписання, але композит у частини може бути лише один, і моделлю ним обрано користувач. Це розходження не приховано, а описано в Розділі 7 і приміткою просто на зв'язку.

Окремо перевірено кратності, бо саме в них найлегше збрехати непомітно. П'ять композицій від користувач мають з боку власника 0..1, а не 1: у схемі рядок може бути ще не прив'язаним до акаунта, і нуль позначає цей перехідний стан. Там, де власник обов'язковий, збережено 1. Агрегація від Матеріал має кратність 1, бо картка слова без матеріалу не створюється взагалі. Асоціація ОцінкаРівня — ВідомеСлово має 0..1 з боку оцінки, бо слово, позначене відомим вручну, з жодного тесту не походить. Кожне з цих рішень читається з моделі й перевіряється одним питанням до схеми даних.

Головним змістовним результатом роботи є не сама діаграма, а три рішення, які вона зробила видимими. Перше — абстрактний клас `Матеріал` з абстрактним методом `джерелоТексту()`: він дозволяє оцінці зрозумілості, збереженню слова й розбору речення працювати однаково з відео й з документом, тому ці сценарії не довелося описувати двічі; нащадків рівно два, бо нащадок з'являється там, де відрізняється реалізація цього методу, а стаття за посиланням дає той самий текст, що й файл. Друге — інтерфейс `джерелоСубтитрів` із фабрикою: він перетворює обмеження «лише YouTube» з вимушеного спрощення на архітектурне рішення про межі роботи, бо нова платформа підключається реалізацією контракту, а не зміною оверлея. Третє — шість класів-довідників без власника: відсутність у них будь-якого зв'язку з `Користувач` одночасно забезпечує приватність (з даних неможливо відновити, хто що дивився) і економію на двох найдорожчих операціях — розпізнаванні мовлення й перекладі доріжки.

Узгодженість із Лабораторною роботою №1 перевірено в трьох зрізах. Усі 18 варіантів використання дістали класи, причому жоден не трасується на клас, відсутній на оцінюваній діаграмі, і всі чотири платні варіанти використання однаково трасуються на гейт підписки. Усі ключові іменники трьох повних текстових специфікацій (UC04, UC05, UC12) знайшли місце серед класів, атрибутів або методів. Відображення акторів на класи свідомо не зроблено взаємно однозначним: актор — це роль, тому «Користувач з підпискою» став не класом-нащадком, а станом `Користувач.тариф()`. Використання ШІ дало корисний чорновий перелік сутностей за кілька хвилин, але всі шість виправлень вимагали знання саме цього сервісу та його схеми даних, а не знання UML: ШІ систематично робить усе частиною головного класу, ставить 1 там, де колонка `nullable`, і дописує атрибути, яких ніхто не зберігає. Побудована модель прямо готує наступні роботи: перелічення `СтанEnum` і `СтанРоботиEnum` уже містять готові набори станів для діаграм станів картки слова й роботи з розпізнавання мовлення, а фактичну, неповну відповідність пакетів класів до восьми компонентів ЛРЗ наведено в Розділі 8.4.